Wrong-Section Report

Thirteen Confirmed Discrepancies, Cross-Referenced to Source

Compiled 2026-07-31

Abstract

This report locates, quotes, and sub-labels every wrong or internally inconsistent section identified in the three manuscript files (`jmlr_paper_main.tex`, `supp_benchmark_report.tex`, `supp_routing_improvements.tex`). Each numbered section below corresponds one-to-one with the issue of the same number in the audit fix list, gives the exact file/label/line location of the conflicting statements, quotes the conflicting text verbatim, and states the category of fix required.

Contents

1 Nguyen-12 Table Success-Count Arithmetic

[STALE — reconciliation only] (partial arithmetic fix) **[OPEN — needs code/data investigation]** (per-equation values unverified)

Location: `jmlr_paper_main.tex`, §`subsec:nguyen12-benchmark-standard-sr-suite` "Nguyen-12 Benchmark (Standard SR Suite)", `tab:nguyen12`, l. 2432–2520.

What is wrong

The table's printed *Success* row and its own per-equation cells disagree with each other under the table's own stated criterion ("Bold: extrap $R^2 \geq 0.9999$"):

> `Success (`$R^2 \geq$`0.9999)  P: 10/12 (83.3%)  H: 11/12 (91.7%)  N: 0/12 (0%)`

A hand recount of the twelve bolded per-equation extrapolation cells gives: **PySR-only (P): 8/12**, not 10/12 (N-3 and N-7 are not ≥ 0.9999 but are not counted as passes, and no other cell was miscounted — the printed total simply does not equal the number of bolded cells). **HypatiaX (H): 7/12**, not 11/12 — two cells (N-3 = 0.9976, N-10 = 0.9997) are typeset in **bold** despite being below the table's own ≥ 0.9999 threshold, and removing them from the bolded/passing count leaves 7, not 11. A third, independently conflicting figure appears in the caption itself:

> `Strict recovery (`≥ 0.9999`) yields 4/12 (33.3%); the 91.7 % figure uses the 4-decimal rounding convention.`

Three different denominators for the same nominal quantity — 10/12 (row), 11/12 (row, also restated in prose at l. 220, 2503), and 4/12 (caption) — appear in the same table environment and do not reconcile against each other or against the twelve visible per-equation cells.

Fix required

The Success row is fixable by hand re-tally against the table's own printed cells. However, per the fix-list's caution, the per-equation R^2 values themselves have not been independently re-verified against the raw result JSON, so a complete fix should regenerate both the per-equation cells and the summary row from source, not merely re-tally the printed numbers.

2 Supp. Table 4 (30/30) vs. Abstract/Conclusion (27/30) on HyperSymLoop (M4)

[STALE — reconciliation only] — classic same-run desync, no code investigation needed

Location: `supp_benchmark_report.tex`, `tab:overall` ("Six-Method Aggregate Performance," l. 394–423) vs. its own Abstract (l. 140–141).

What is wrong

The Abstract states:

> HyperSymLoop achieves 100% recovery at all *noisy* conditions ($\sigma > 0$) and 90.0% (27/30 equations) under the strict noiseless protocol ($R^2 > 0.9999$).

But `tab:overall`, in the same document, prints:

`HyperSymLoop (M4)  Core  30/30 (v2)†  0.999+  11.4 s`

with a footnote reading "HyperSymLoop v1 showed 26/30 due to the Newton measurement bug; v2 achieves 30/30." The table's 30/30 and the abstract's 27/30 are both presented as the current (v2, corrected) figure for the same method on the same noiseless protocol, in the same document.

Fix required

Regenerate `tab:overall`'s HyperSymLoop row from whichever run actually produced the 27/30 figure quoted in the abstract, and confirm both numbers come from the same run before republishing either.

3 EHSDeFi (M3) Runtime: 20.2 s vs. 841.4 s

[OPEN — needs code/data investigation] — needs live code/data investigation

Location: `supp_benchmark_report.tex`, `tab:overall` l. 413 vs. `tab:time_noise` ("Sweep Results: Noise Robustness") l. 571–584.

What is wrong

`tab:overall` reports EHSDeFi's (M3) average runtime as:

`EHSDeFi (M3)  Core  29/30 (96.7%)  0.999993  20.2 s`

`tab:time_noise`, covering the same method at $\sigma = 0\%$ (the same noiseless condition), reports:

`0%  841.4  11.1  75.8×`

The abstract's own claim that "EHSDeFi offers exact symbolic interpretability; HyperSymLoop is 1.5–76× faster depending on noise level" corroborates the 841.4 s figure (75.8× vs. M4's 11.1 s), not the 20.2 s figure.

Fix required

Trace which script/run produced 20.2 s. If it reflects different hardware/config, that must be stated explicitly; if it is simply wrong, it must be regenerated from the same raw timing logs that produced `tab:time_noise`.

4 CI / SD Statistical Incompatibility

[OPEN — needs code/data investigation] — needs live code/data investigation

Location: `jmlr_paper_main.tex`, §`subsec:fivesystem-comparative-analysis-feynman-` "Five-System Comparative Analysis (Feynman Core-15)," `tab:five_systems_full` and `thm:five_system_hierarchy`, l. 1213–1259 vs. `supp_benchmark_report.tex`, Appendix G "Statistical Test Details," `app:statistical_tests`, l. 1423–1471.

What is wrong

The main paper's Result box states:

> `neural networks exhibit mean error 1231% (95% CI: [1087%, 1456%],` $n = 13$`)`

That gives a CI half-width of ≈ 185 percentage points around a mean of 1231%. Appendix G, describing the same neural-network distribution, states:

> `Neural Network:  Mean = 1231%, Median = 86.7%, SD = 1842%, Range = [12.3, 5847%] (CORRECTED)`

A standard 95% CI on the mean with $n = 13$ and SD = 1842% has a half-width of roughly $t_{0.975,12} \cdot 1842/\sqrt{13} \approx 2.18 \times 511 \approx 1114$ percentage points — almost six times narrower a CI is reported in the main text than the disclosed SD supports.

Fix required

Re-run whatever CI computation produced Table 1's figure and check it against Appendix G's SD; the CI was very likely computed with the wrong SD, wrong n, or an undisclosed non-standard method.

5 "68 of 74" Cited Without Seed/Split

[TEXT — pure writing fix] — pure writing/citation fix, underlying data is consistent

Location: `jmlr_paper_main.tex`, Abstract l. 222–223 and §`subsec:fivestage-routing-architecture-overvi` l. 1011, referencing Table `tab:timing_full` (§`subsec:runtime-analysis`).

What is wrong

Both the abstract and §`subsec:fivestage-routing-architecture-overview` cite "68 of 74" LLM-routed tasks with no seed or split attached:

> `A previously reported 1.73× median speedup of HypatiaX over neural-network inference on LLM-routed cases (68 of 74) does not reproduce...`
>
> `Stage 3 (optional):  LLM proposes candidate expressions; on the v3.0 DeFi benchmark, 68 of 74 tasks are routed to this path.`

But Table `tab:timing_full` shows *two different* rows with exactly 68/74 LLM-routed tasks — `v3c seed123` (68/74) and `PCA seed42` (68/74) — which are different seeds under different splits with different absolute runtimes. "68 of 74," as written, does not identify which of the two.

Fix required

Attach the specific seed and split (e.g. "68 of 74 under v3c seed 123" or "... PCA seed 42") wherever "68 of 74" is cited. No table value needs to change.

6 "v3c" vs. "PCA-directed" Never Defined as Two Variants

[TEXT — pure writing fix] — pure writing fix, both variants' numbers are real

Location: `jmlr_paper_main.tex`, §`subsec:extrapolation-protocol` (`sec:split`, l. 678–709) vs. Table `tab:timing_full` / Table `tab:timing_llm_routed_full` captions (l. 1630–1636).

What is wrong

§subsec:extrapolation-protocol ("Extrapolation Protocol") defines exactly *one* split methodology:

> To rigorously assess out-of-distribution generalisation, we employ a PCA-directed aggressive extrapolation split: ...

No "v3c" variant is introduced or distinguished anywhere in this section. Yet Table tab:timing_full's caption (and every runtime table downstream) treats v3c and PCA as two distinct, sibling splits:

> Rows are grouped by split (v3c = aggressive 40/60; PCA = PCA-directed 40/60)...

A reader following §subsec:extrapolation-protocol alone has no way to learn that "v3c" and "PCA-directed" name two different, non-identical protocols rather than being two names for the same one.

Fix required

Add an explicit definition of both variants (what specifically distinguishes v3c's "aggressive" routing from the PCA-directed routing already described) to §subsec:extrapolation-protocol, since the underlying runs for both are real.

7 Remark on Difficulty-Tier Counts: 24/27/20 → 24/29/21 vs. "+3 Hard"

[TEXT — pure writing fix] — pure text/arithmetic fix

Location: jmlr_paper_main.tex, §subsec:difficulty-classification unlabeled remark following the Easy/Medium/Hard definitions, l. 656–660.

What is wrong

The tier definitions immediately above state Easy $n = 24$, Medium $n = 29$, Hard $n = 21$ (totalling 74). The remark directly below reads:

> An earlier version of this benchmark comprised 71 tasks (Easy=24, Medium=27, Hard=20). The current benchmark (v3.0, 74 tasks) adds three additional hard cases in multi-asset risk: correlated portfolio VaR, component ES, and multi-collateral LTV.

$71+3 = 74$ is correct in total, but 24/27/20 → 24/29/21 is a change of +0/+2/+1, not +0/+0/+3. The remark's claim that all three additions are Hard-tier is arithmetically inconsistent with the tier counts stated one paragraph above it.

Fix required

Either the remark's "adds three additional hard cases" is wrong (should read something like "+2 Medium, +1 Hard"), or the Medium/Hard counts in §subsec:difficulty-classification are wrong. Purely a text correction; no code or data investigation implicated.

8 Figure 3 Caption "DeFi AMM (Hard) = −78.04" Untraceable, Plus Impermanent-Loss Tier Clash

[OPEN — needs code/data investigation] — confirmed known-bad figure/caption, fix source already identified

Location: `jmlr_paper_main.tex`, Figure `fig:r2_heatmap_clipped` caption (`fig18_r2_heatmap_improved`, l. 1287–1300) vs. Table `tab:llm_ablation` (l. 1871–1897) vs. the Medium-tier definition (l. 632–634).

What is wrong

The Figure 3 caption reads:

> `HypatiaX is not uniformly better than PySR-only: it improves on Chemistry (Easy:` 0.90 `vs.` −12.56`) but produces` −∞ `-- a genuine evaluation crash, not merely a poor fit -- on DeFi AMM (Hard) and Physics (Hard), where PySR-only instead returns a finite (if poor,` −78.04`, or perfect,` 1.00`) value.`

Cross-checking against Table `tab:llm_ablation` (explicitly stated to underlie this figure): no "DeFi AMM" equation shows PySR-only = −78.04 at any range (Constant Product P: 0.9982–0.9996; Impermanent Loss P: 0.7703 near, −2.7832 med, −157.0776 far; Price Impact P: all ≈ 1.0). The −∞ HypatiaX crash described for the DeFi AMM row also does not appear in the DeFi AMM rows of Table `tab:llm_ablation` (all DeFi AMM H values are finite, if very large-magnitude negative); the only −∞ H value in the whole table belongs to **Gravitational Force**, which is Physics, not DeFi AMM. Separately, the caption implies a DeFi AMM task sits in the **Hard** tier, but the difficulty-tier definition explicitly lists Impermanent Loss — the DeFi AMM equation actually exhibiting catastrophic values in Table `tab:llm_ablation` — as a **Medium**-tier example:

> `Medium (`$n = 29$`): nonlinear algebraic or exponential relationships, such as impermanent loss, Uniswap V3 liquidity range, constant-product price impact...`

Fix required

Regenerate Figure 3 from the same `_merged.json` data used to rebuild Table `tab:llm` $_{a}blation, ratherthanwhateverprocessproducedthecaption's$`-78.04/`$DeFi-AMM-Hardclaims, andresolvewheth Loss-classtaskbelongstoMediumorHardconsistentlyacrossthetierdefinitionandthefigure.$

9 Uncorrected Mean R^2 (+0.8721) Despite Disclosed −141,000 Masked Failure

`[OPEN -- needs code/data investigation] -- pending, not yet executed against a full live re-run`

Location: `jmlr_paper_main.tex, §subsec:overall-extrapolation-performance-defi-v tab:main_results, l. 1261-1285, and the Abstract's masking-disclosure footnote, l. 209-215.`

What is wrong

`The abstract discloses that HypatiaX's own success accounting silently masks catastrophic sub-method failures:`

> The underlying computation is not free of catastrophic failures: 22 of the 74 tasks route to a sub-method (typically the LLM proposal) that itself failed catastrophically (e.g. pure_llm.test_r2 $\approx -141{,}000$ on Loan-to-Value), but the hybrid system's own accounting reports success=True, $R^2 \approx 1.0$ regardless. The catastrophic failure is masked, not eliminated.

Yet tab:main_results, whose own caption acknowledges the 90.5% pass-rate figure is uncorrected for this exact bug, still reports HypatiaX's Mean R^2 as $+0.8721$ -- computed, like the pass rate, from the same masked success/R^2 fields that report ≈ 1.0 for the 22 tasks whose true sub-method result is catastrophic:

> HypatiaX 1.0000 $+0.8721$ 90.5 90.5 0

The caption corrects the pass-rate columns ("the corrected overall near-perfect rate is 60.8%") but is silent on the Mean R^2 column, which is left standing as if unaffected by the same bug.

Fix required

Recompute Mean R^2 from the corrected (decision-attribution-fixed) full live run described in §sec:hybrid-attribution-bug; this is explicitly stated elsewhere as not yet executed, so the corrected number cannot be hand-estimated from the single disclosed $-141{,}000$ outlier alone.

10 Timeout Contradiction: 1100 s vs. 900 s; "300 s Limit" but a 27,574 s Hang

[OPEN -- needs code/data investigation] (enforcement bug, second half) / [STALE -- reconciliatio only] (config bleed, first half)

Location: supp_benchmark_report.tex, Sweep hyperparameters (tab:sweeps, l. 294-311) vs. Reproducibility Statement, Phase 3 reproduction command, l. 988-1004.

What is wrong

tab:sweeps' hyperparameter note states:

> Hyperparameters: NN seeds = 5, random seed 42, method timeout = 900 s, PySR timeout = 1100 s, threshold (noisy) = 0.995, threshold (noiseless) = 0.999999. All 30 equations completed with zero timeouts across all 11 conditions.

The Phase 3 reproduction command block, describing (per its own caption) the run underlying tab:overall, instead shows:

```
python run_comparative_suite_benchmark_v2.py \
-noiseless -threshold 0.9999       \
-nn-seeds 3 -samples 200           \
-method-timeout 900                     \
-pysr-timeout 900
```

i.e. 900 s for *both* timeouts, not 1100 s for PySR as tab:sweeps states. Immediately below, the same reproducibility note directly contradicts the "zero timeouts" claim:

> Note on timeout upgrade: -method-timeout was set to 300 s for tests 1-18 and upgraded to 900 s for tests 19-30. Arrhenius (test 4) ran under the 300 s limit and hung for 27574 s before the Julia process was killed...Nernst (test 6) also timed out under the 300 s default; counted as failure.

A stated "300 s limit" allowing a process to run for 27,574 s (over 90× the configured limit) before manual kill indicates the timeout was not actually enforced for that run.

Fix required

First half (1100 s vs. 900 s): reads as two different phases' configs bleeding into one write-up -- reconcile which value is correct for tab:sweeps and state it consistently. Second half (300 s limit, 27,574 s hang): the timeout *enforcement* itself needs checking in the harness code (or was silently disabled for that run) -- this is a code-level question, not just a documentation fix.

11 Model/Temperature Mismatch Across Documents

[ALREADY RESOLVED -- doc propagation only] -- root cause already found; only stale doc entries remain

Location: supp_benchmark_report.tex, reproducibility tables l. 1045-1046 and l. 1243/1261, and supp_routing_improvements.tex, "Hardware and Software" l. 756, vs. jmlr_paper_main.tex, Nguyen-12 reproducibility note, l. 2606-2637.

What is wrong

jmlr_paper_main.tex already contains the corrected finding: the model that actually produced the Nguyen-12 results is claude-sonnet-5 at temperature=0.25, via hypatia.py -- confirmed as a fourth, distinct hardcoded string, with the LLM_MODEL environment variable and repro.yaml's llm_model field both confirmed to be dead code that nothing reads. Three stale locations in the supplementary files still report the superseded value:

> supp_benchmark_report.tex:1045 Anthropic SDK 0.73.0 (claude-sonnet-4-20250514, temperature 0.0)

> supp_benchmark_report.tex:1243, 1261 Model: claude-sonnet-4-20250514 LLM temperature: 0.3

> supp_routing_improvements.tex:756 Anthropic SDK 0.73.0 claude-sonnet-4-20250514, temp 0.0

Fix required

Overwrite all three stale entries with the confirmed value (claude-sonnet-5, temperature=0.25). No further investigation needed -- this is doc propagation only.

12 Rounding Slippage: $1.21/0.36 = 3.36\times$, Printed $3.40\times$

[OPEN -- needs code/data investigation] -- narrow, mechanical script fix + regen

Location: jmlr_paper_main.tex, §subsec:runtime-analysis Table tab:timing_full pooled PCA row, l. 1636.

What is wrong

The pooled PCA row of Table tab:timing_full reads:

> PCA ALL SEEDS (pooled) 370 2.41 / 0.18 0.36 / 0.35 1.21 / 0.89 94/370 3.40× slower

Dividing the printed Hybrid mean (1.21 s) by the printed Neural MLP mean (0.36 s) gives $1.21/0.36 \approx 3.361\times$, not the printed $3.40\times$. (By contrast every v3c-split row's printed speedup is internally consistent with its own printed mean columns.)

Fix required

generate_table1.py (already named in the paper as the regeneration script) is almost certainly dividing pre-rounded display values instead of full-precision ones; needs a one-line fix plus a full table regeneration.

13 Near-Duplicate Task Names Across Difficulty Tiers

[TEXT -- pure writing fix] -- naming-clarity issue, not a data bug

Location: jmlr_paper_main.tex, §sec:hybrid-attribution-bug fabricated-success task list, l. 1540-1550, vs. §subsec:lending-protocols "Lending Protocols" task inventory, l. 2960-2977.

What is wrong

The Lending Protocols task inventory lists two visually near-identical pairs of tasks as distinct benchmark items:

> Liquidation Price Long and Liquidation Price Short: prices at which a position is liquidated. ...Liquidation price for leveraged long and leveraged short: extended formulas accounting for funding rates.

Both pairs then appear, split across different difficulty tiers, in the fabricated-success task list:

> Medium (10): ...Liquidation Price Long, Liquidation Price Short, ...
>
> Hard (9): ...Liquidation price for leveraged long, Liquidation price for leveraged short, ...

A reader skimming either list alone could easily mistake "Liquidation Price Long" (Medium) for "Liquidation price for leveraged long" (Hard), or assume a duplicate/copy-paste error, when in fact these name four formulas that are genuinely distinct (the leveraged variants explicitly account for funding rates).

Fix required

Rename one pair for clarity (e.g. "Liquidation Price, Basic Long/Short" vs. "Liquidation Price, Leveraged Long/Short (funding-adjusted)") so the two are visually distinguishable wherever they are listed. No numeric or code change needed.

Summary Table

#	Issue	Fix category	Primary files
1	Nguyen-12 success-count arithmetic (10/12, 7-actual, 4/12)	Stale + Open	jmlr_paper_main
2	Supp Table 4 (30/30) vs. Abstract (27/30), M4	Stale	supp_benchmark_report
3	EHSDEFI runtime 20.2 s vs. 841.4 s	Open	supp_benchmark_report
4	CI/SD statistical incompatibility	Open	jmlr_paper_main / supp_benchmark_report
5	"68 of 74" without seed/split	Text	jmlr_paper_main
6	v3c vs. PCA-directed undefined	Text	jmlr_paper_main
7	Remark 9 tier arithmetic (+3 Hard)	Text	jmlr_paper_main
8	Fig. 3 caption -78.04 untraceable + tier clash	Open (source ID'd)	jmlr_paper_main
9	Uncorrected Mean R^2 despite masked failure	Open (pending re-run)	jmlr_paper_main
10	Timeout contradiction (1100 vs. 900 s; 300 s limit / 27,574 s hang)	Open + Stale	supp_benchmark_report
11	Model/temperature mismatch	Already resolved (doc only)	supp_benchmark_report / supp_routing_improvements
12	Rounding slippage 3.36× vs. 3.40×	Open (mechanical)	jmlr_paper_main
13	Near-duplicate task names	Text	jmlr_paper_main